

Contents

Product Backlog	1
Epic 1 — Bewegung & Überleben	2
Epic 2 — Angriff	5
Epic 3 — Strategie & Zustände	7
Epic 4 — Qualität (optional)	10
Epic 5 — Turniervorbereitung	11

Product Backlog

Gemeinsames Backlog für alle drei Teams. Jede Story hat Akzeptanzkriterien und Story Points (Planning-Poker-Skala 1/2/3/5 — 1 = trivial, 5 = anspruchsvoll für diese Gruppe). Reihenfolge der Epics ist eine Empfehlung, keine Pflicht-Sortierung.

Für jede Story gibt es eine Musterlösung für den Dozenten in [dozent/loesungen.md](#) — bitte nicht an Schüler weitergeben, bevor sie es selbst versucht haben.

Für Schüler: Denkhilfen, Taktik-Gedanken und Leitfragen zu jeder Story (mit Bildern) stehen in [story-hinweise.md](#). Dort nachschauen, wenn ihr an einer Story hängt.

Referenz-API (siehe auch `framework/arena/Models.kt`):

```
interface RobotBrain {
    val name: String
    fun decide(sensors: Sensors): Action
}
// sensors.self: eigener RobotState (position, health, alive)
// sensors.others: alle anderen lebenden Roboter
// sensors.arenaWidth / arenaHeight: Rastergröße (10x10)
// sensors.tick: aktueller Tick-Zähler (ab 0)
// Action: Move(Direction) | Shoot(Direction) | Wait
// Direction: NORTH, EAST, SOUTH, WEST
```

Für Distanz-, Richtungs- und Gegnersuche-Berechnungen (nächster Gegner, Fluchtrichtung, Sichtlinie, Rand/Mitte-Erkennung, ...) gibt es fertige Helferfunktionen — selbst herrechnen ist nicht nötig. Volle Übersicht: [toolkit-referenz.md](#).

Epic 1 — Bewegung & Überleben

Story 1.1 — Zufällige Bewegung (Warmup)

Story Points: 1

Als Team möchte ich, dass mein Bot sich bei jedem Tick in eine zufällige Richtung bewegt, damit ich sehe, dass mein Bot überhaupt korrekt ins Spiel eingebunden ist.

Akzeptanzkriterien:

- `decide()` gibt bei jedem Aufruf `Action.Move(direction)` mit einer zufällig gewählten `Direction` zurück.
- Der Bot ist in der Bot-Auswahl der App sichtbar und bewegt sich sichtbar über die Arena.

Story 1.2 — Am Rand bleiben / Zentrum meiden

Story Points: 1

Als Team möchte ich, dass mein Bot sich eher am Rand der Arena aufhält (schwerer zu treffen, wenn weniger Gegner in Reihe/Spalte kommen), damit er nicht sofort im Zentrum kampiert und leicht von mehreren Seiten getroffen wird.

Akzeptanzkriterien:

- Der Bot berücksichtigt `sensors.self.position` relativ zu `sensors.arenaWidth/arenaHeight`.
- Befindet sich der Bot zu nah am Zentrum, bewegt er sich in Richtung eines Randes.

Story 1.3 — Flucht bei niedriger Gesundheit

Story Points: 2

Als Team möchte ich, dass mein Bot flieht, wenn seine Gesundheit unter 20 fällt, damit er nicht sinnlos weiterkämpft und vielleicht überlebt.

Akzeptanzkriterien: - Der Bot findet den nächsten Gegner. - Ist `sensors.self.health < 20`, bewegt sich der Bot vom nächsten Gegner weg (nicht auf ihn zu). - Ist `sensors.self.health >= 20`, verhält sich der Bot wie zuvor (normale Logik, z.B. aus Epic 2). - Gibt es keinen Gegner mehr, wirft der Bot keine Exception (z.B. `Action.Wait` als Fallback).

Story 1.4 — Keine Bewegung verschwenden (Randcheck)

Story Points: 1

Als Team möchte ich, dass mein Bot keine Bewegung in Richtung außerhalb der Arena vorschlägt, damit er nicht durch einen ungültigen Zug einen Tick verschenkt.

Akzeptanzkriterien: - Vor der Bewegung prüft der Bot, ob das Zielfeld der gewählten Richtung überhaupt in der Arena liegt. - Würde die gewählte Richtung aus der Arena hinausführen, wählt der Bot stattdessen eine andere, gültige Richtung. - Der Bot bewegt sich dadurch nie mehrere Ticks hintereinander ergebnislos "gegen die Wand".

Story 1.5 – Sicherheitsabstand zum nächsten Gegner halten

Story Points: 2

Als Team möchte ich, dass mein Bot unabhängig von seiner Gesundheit einen Mindestabstand zum nächsten Gegner hält, damit er nicht ungewollt direkt neben einem Gegner landet.

Akzeptanzkriterien: - Der Bot berechnet die Distanz zum nächsten Gegner. - Unterschreitet die Distanz einen selbst gewählten Schwellenwert (z.B. 2 Felder), bewegt sich der Bot vom Gegner weg. - Ist der Abstand groß genug, verhält sich der Bot normal (z.B. Zufallsbewegung aus 1.1). - Gibt es keinen Gegner mehr, wirft der Bot keine Exception.

Story 1.6 – Fluchtrichtung mit Wandvermeidung kombinieren

Story Points: 2

Als Team möchte ich, dass mein fliehender Bot nicht gegen die Arena-Wand rennt, damit die Flucht aus Story 1.3 auch am Rand der Arena zuverlässig funktioniert.

Akzeptanzkriterien: - Würde die Fluchtrichtung aus der Arena hinausführen, weicht der Bot stattdessen in eine andere gültige Richtung aus. - Weit weg vom Rand verhält sich der Bot wie in Story 1.3. - Gibt es keinen Gegner mehr, wirft der Bot keine Exception.

Epic 2 — Angriff

Story 2.1 — Dauerfeuer in feste Richtung

Story Points: 1

Als Team möchte ich, dass mein Bot ständig in eine feste Richtung schießt, damit ich die grundlegende Schuss-Mechanik ausprobieren kann.

Akzeptanzkriterien:

- `decide()` gibt immer `Action.Shoot(direction)` mit einer festen Direction zurück.
- Im Testduell gegen `StillstandBot` (aus `bots/examples`) wird sichtbar Schaden verursacht, wenn die Ausrichtung passt.

Story 2.2 — Auf nächsten Gegner in Sichtlinie zielen

Story Points: 2

Als Team möchte ich, dass mein Bot gezielt auf einen Gegner schießt, der sich exakt in seiner Reihe oder Spalte befindet, damit Treffer nicht dem Zufall überlassen sind.

Akzeptanzkriterien:

- Der Bot findet den nächstgelegenen Gegner, der exakt in seiner Reihe oder Spalte steht.
- Steht kein Gegner in Sichtlinie, tut der Bot etwas anderes (z.B. bewegen, siehe Story 2.3) statt sinnlos ins Leere zu schießen.

Story 2.3 — Gegner verfolgen, wenn nicht in Schusslinie

Story Points: 2

Als Team möchte ich, dass mein Bot einem Gegner hinterherläuft, wenn er ihn nicht direkt anvisieren kann, damit er aktiv Kämpfe sucht statt nur zu warten.

Akzeptanzkriterien:

- Steht kein Gegner in Sichtlinie (siehe 2.2), bewegt sich der Bot einen Schritt in Richtung des nächstgelegenen Gegners.
- Kombiniert mit Story 2.2: steht ein Gegner in Sichtlinie -> schießen, sonst -> verfolgen.

Story 2.4 — Nicht ins Leere schießen

Story Points: 1

Als Team möchte ich, dass mein Bot nur schießt, wenn er wirklich ein Ziel treffen kann, damit er nicht sinnlos einen Tick mit einem Schuss ins Leere verschwendet.

- Akzeptanzkriterien:**
- Vor jedem `Action.Shoot(...)` prüft der Bot, ob überhaupt ein Gegner in der gewählten Richtung steht.
 - Kann kein Gegner getroffen werden, tut der Bot stattdessen etwas anderes (z.B. bewegen, siehe 1.1/2.3) statt zu schießen.
 - Gibt es keinen Gegner mehr, wirft der Bot keine Exception.

Story 2.5 — Schwächsten Gegner in Sichtlinie zuerst angreifen

Story Points: 2

Als Team möchte ich, dass mein Bot unter mehreren Gegnern in Sichtlinie gezielt den mit den wenigsten HP angreift, damit er Gegner schneller ausschaltet statt wahllos zu schießen.

- Akzeptanzkriterien:**
- Der Bot ermittelt alle in Sichtlinie treffbaren Gegner und wählt darunter den mit der niedrigsten `health` als Ziel.
 - Ist kein Gegner in Sichtlinie, verhält sich der Bot wie in Story 2.3 (verfolgen).
 - Gibt es keinen Gegner mehr, wirft der Bot keine Exception.

Story 2.6 — Rückzug beim Schießen vermeiden

Story Points: 2

Als Team möchte ich, dass mein Bot beim Verfolgen eines Gegners (Story 2.3) nicht direkt neben ihm stehen bleibt, damit er nicht unnötig nah am Gegner steht, sobald er ihn treffen kann.

Akzeptanzkriterien: - Steht der Bot in Sichtlinie zum nächsten Gegner, aber näher als ein selbst gewählter Mindestabstand (z.B. 1 Feld), bewegt er sich einen Schritt zurück, statt weiter anzugreifen. - Ist der Abstand groß genug, schießt der Bot wie in Story 2.2. - Steht kein Gegner in Sichtlinie, verfolgt der Bot wie in Story 2.3.

Epic 3 — Strategie & Zustände

Story 3.1 — Einfache Zustandsmaschine (Patrouille / Angriff / Flucht)

Story Points: 5

Als Team möchte ich, dass mein Bot zwischen mehreren klar benannten Verhaltenszuständen wechselt, damit sein Verhalten nachvollziehbar und erweiterbar ist.

Akzeptanzkriterien: - Mindestens 3 Zustände sind erkennbar (z.B. PATROUILLE wenn kein Gegner in der Nähe, ANGRIFF wenn ein Gegner erreichbar ist, FLUCHT wenn `health < 20`). - Der Zustand wird bei jedem `decide()`-Aufruf neu aus den aktuellen `sensors` bestimmt (kein gespeicherter Zustand nötig, aber erlaubt — z.B. über eine `var` im Bot). - Jeder Zustand führt zu klar unterscheidbarem, im Log sichtbarem Verhalten.

Story 3.2 – Zielpriorisierung bei mehreren Gegnern

Story Points: 2

Als Team möchte ich, dass mein Bot bei mehreren möglichen Zielen sinnvoll auswählt (z.B. den schwächsten oder nächsten Gegner), damit er nicht wahllos das erste beste Ziel angreift.

Akzeptanzkriterien: - Bei mehreren Gegnern in `sensors.others` wählt der Bot nach einem klaren Kriterium (z.B. niedrigste `health` oder kleinste Distanz). - Das Kriterium ist im Code klar benannt/kommentiert.

Story 3.3 – Kür-Aufgabe (freie Wahl)

Story Points: 5

Als Team möchte ich eine eigene Idee umsetzen, die über die vorgegebenen Storys hinausgeht, damit ich meine eigene Kreativität einbringen kann.

Akzeptanzkriterien: - Team formuliert die Story selbst (kurz, 1-2 Sätze) und trägt sie als eigene Karte ins Board ein. - Dozent bestätigt kurz, dass die Idee mit der bestehenden `RobotBrain-API` umsetzbar ist, bevor das Team startet. - Beispiele: Bewegungsmuster, das Gegner in eine Ecke drängt; Bot, der sich an der Wand entlang bewegt; einfache “Ich schieße nur, wenn ich sicher treffe”-Heuristik.

Story 3.4 — Zeitgesteuerte Startphase (Patrouille zuerst)

Story Points: 2

Als Team möchte ich, dass mein Bot in den ersten Ticks erst patrouilliert statt sofort anzugreifen, damit er nicht direkt zu Matchbeginn unüberlegt in einen Kampf läuft.

Akzeptanzkriterien:

- Solange `sensors.tick` unter einem selbst gewählten Schwellenwert liegt (z.B. 10), bewegt sich der Bot nur (z.B. wie in Story 1.1 oder 1.2), auch wenn ein Gegner sichtbar ist.
- Ab dem Schwellenwert verhält sich der Bot wie die bisherige Angriffslogik (z.B. aus Epic 2 oder Story 3.1).
- Der Schwellenwert ist im Code als klar benannte Konstante erkennbar.

Story 3.5 — Abklingzeit nach der Flucht

Story Points: 3

Als Team möchte ich, dass mein Bot nach einer Flucht noch einige Ticks vorsichtig bleibt, damit er nicht sofort nach knappem Entkommen wieder ungeschützt in den nächsten Kampf läuft.

Akzeptanzkriterien:

- Der Bot merkt sich (z.B. über eine `var`-Property in der Bot-Klasse) für wie viele Ticks er sich zuletzt im Flucht-Zustand befunden hat.
- Für eine selbst festgelegte Anzahl Ticks nach dem Ende einer Flucht verhält sich der Bot weiter vorsichtig (z.B. Rand halten statt angreifen).
- Nach Ablauf dieser Zeit verhält sich der Bot wieder wie zuvor (z.B. Zustandsmaschine aus Story 3.1).

Epic 4 — Qualität (optional)

Story 4.1 — Unit-Test für eigene Entscheidungslogik

Story Points: 2

Als Team möchte ich einen einfachen Unit-Test für einen Teil meiner `decide()`-Logik schreiben, damit ich das in der Praxis ausprobiert habe.

Akzeptanzkriterien:

- Mindestens ein Test in `src/test/kotlin/bots/teamX/...` (Ordner ggf. neu anlegen) ruft `decide()` mit einem selbst gebauten `Sensors`-Objekt auf und prüft das Ergebnis mit `assertEquals/assertTrue`.
- Test ist grün (`./gradlew test`).

Story 4.2 — Testduell protokollieren

Story Points: 1

Als Team möchte ich ein Testduell gegen einen Beispiel-Bot durchführen und das Ergebnis kurz notieren, damit wir den Fortschritt unseres Bots dokumentieren.

Akzeptanzkriterien:

- Testduell gegen mindestens einen Bot aus `bots/examples` durchgeführt.
- Ergebnis (Sieg/Niederlage/Unentschieden, ungefährender Verlauf) in 1-2 Sätzen notiert (z.B. auf der Board-Karte oder in einer Notiz-Datei im Team-Ordner).

Epic 5 – Turniervorbereitung

Story 5.1 – Finale Bot-Version festlegen

Story Points: 1

Als Team möchte ich am Ende von Tag 3 eine klar benannte, funktionierende Bot-Version für das Turnier bereithaben, damit die Integration beim Dozenten reibungslos läuft.

Akzeptanzkriterien: - teamXBots-Liste in TeamXBots.kt enthält genau die Bot-Instanz(en), die im Turnier antreten sollen. - Projekt kompiliert fehlerfrei (`./gradlew build`).

Story 5.2 – Strategie-Kurzvorstellung

Story Points: 1

Als Team möchte ich unsere Bot-Strategie in 1-2 Sätzen vorstellen können, damit die anderen Teams und der Dozent beim Review verstehen, was unser Bot tut.

Akzeptanzkriterien: - Team kann in eigenen Worten erklären, wann ihr Bot angreift, flieht oder patrouilliert.